For bkhack, the user gets to familiarize with the concept of *stream-based programming*. Indeed, visitors of the site will often catch glance of command hints and tips as they are littered about the user interface. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

Stream programming in bkhack is sh pipelining [She], with certain flavorful differences[1]. The most important difference is that, instead of dealing with textual data and lines and files, here we deal with abstract post data and post counts. Consider

```
feed | split -c 10
```

where `split` has the option `-c NUM` where `NUM` is the count size of each chunk. Conventionally, the POSIX `split` command accepts the option `-l NUM` where `NUM` is line number size of chunk.

Another difference is that there is no filesystem, no concept of "space". There is a command for every page, but there's no way to navigate "relatively" such as going "back" or "forth"; in other words, there's no `cd` command.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1. Semantics

The original sh language is a user interface for the system shell, which itself is an interface for the kernel, as part of the shell-kernel architecture [AT&]. In comparison, the bkhack shell language is highly abstract and doesn't need a stand-in kernel analogy: at most, the bkhack website's command system attempts to emulate the shell-kernel architecture, so that the user feels the shell-kernel architecture even when it's not really there.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

2. Session

A shell is usually seen as a session that has REPL behavior, an environment with its own variables, and shells can stack on top of one another and can be "popped".

For bkhack, Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri.

3. Composition via pipeline

Introduced by Douglas McIlroy, pipelining enables commands to compose with each other parsibly simply via textual data. In this composition, there are relationships between commands to form the pipeline by parts. This composition is part of the design patterns in designing Unix programs as a whole [Ray03].

The *source* e.g. `ls`, `cat`, etc is the start of the pipeline, it exclusively produces output. The *sink* is the end of the pipeline, it exclusively consumes the input. In-between are *filters* e.g. `grep`, `sort`, `split`, `cut`, etc where data are passed in-and-out and get transformed in the process[2]. So, a simple pipeline is one with a structure source-filter-sink. A complex pipeline is one that can "branch" ; the way to branch is to organize each pipeline branch as each group of commands as in §4, then use special filters to branch on conditions using redirections with named pipes[3]. The rest are *cantrips* e.g. `mv`, `rm`, etc–commands that don't produce output but instead apply an effect onto the current environment.

It's worth noting that, even on the conceptual level, the evaluation order of the part commands

[1] "flavorful" here means that the differences are poignant ones that the language writer feels when she transitions between the languages. See more at XXX

[2] in the context of a pipeline-based system architecture, these are called *middle-wares*

[3] in practice, this is rarely done

is that all commands start simultaneously when a pipeline runs. For a pipeline that runs persistently, all pipes must run concurrently and persistently as well. This is contrary to the semantics of pipeline seen elsewhere e.g. functional programming.

For the bkhack shell language, pipelining is first-class citizen. The grammar expects all commands to unequivocally form a pipeline, as evident by Table 1, and multiple groups of commands will connect to form complex pipelines that branch.

4. Composition via function

A reusable pipeline or grouping of commands would be called a *function*. In the bkhack shell language, Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos.. Function is a useful abstraction. For example, since feed can be treated as a function simply built-in, it's possible to customize the behavior of `feed` by overloading it. Indeed, the default `feed` command in bkhack, which automatically has limiting of 15 items pagination, is simply a function

```
feed() { feed | split -c 15 | cut -f1 |
sort --hot ;}
```

which the user can customize. The rest of bkhack shell commands are exposed like so, thus the settings system.

5. Syntax

Sometimes, it is need to parse the shell language from a text. The grammar of the bkhack shell language, given in EBNF form in Table 1, Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri. Notice how the command rule do not distinguish parts into flags or values which are expected from most real-world usage. Indeed, the responsibility of interpreting these parts, a.k.a. *options*, is left to the higher-level *option parser*, such as POSIX `getopts`.

5.1. Parsing

Parsing is implemented based on the grammar at Table 1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri.

In the initial design, we have only one phase for our parser. This is an eager, AST-less, final a.k.a. *syntax-directed* parser–effects are eagerly executed as each term is parsed. Eventually, we did make it so that it outputs an AST, for reusability and convenience.

<program>	::=	<command> (| <program>)?	*Simple program*
	|	{ <program> <end of seq> } (| <program>)?	*Composite program*
	|	(<program>) (| <program>)?	*Subshell*
<command>	::=	<word> (<command>)?	
<end of seq>	::=	;	
<word>	::=	*Omitted*	
<seq connective>	::=	;	*Default continuation*
	|	&&	*Success continuation*

Table 1: Grammar for the bkhack shell language

In the next design, we have two phases for our parser. The lexical analysis phase (bộ phân tích từ vựng [Pha])[4] and the parsing phase. The lexical analysis phase is to guarantee a minimal schema for the parsers so that they won't diverge from the language spec. This is an implicit structure. In contrast, we could have defined an ADT or GADT to make an explicit structure. However, our parsers won't be final, we would make a compromise on performance. Not only that, an implicit structure allows room for undefined behaviors; each parser can do things in flexible ways that should hopefully achieve desirable results.

Indeed, the lexing phase was added when we branched the parser to support *pastelling*; so, two parsers, a parser and a pastel. A shared lexing phase helps communicating that the two parsers share a structure, even if that structure is only infra-, and ter'es no explicit GADT or signature to explicitly enforce it.

The usefulness of this starts to show when we added a new feature: support for quoted words. Thanks to the two-phase separation, we managed to implement this feature simply by modifying the lexer. We added new parsing rule, and changed the return type of $<$ word $>$. This cascades into all descendant parsers, requiring them to handle the new feature. Here, they are being rewritten by adding a new $<$ word $>$ overloading guard before usage.

6. Typing

Sometimes, it is useful to verify the shell language before evalutation. The typing of the bkhack shell language, given in Figure 1, Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri. This is useful when shell commands have to be embedded in a statically-typed programming language.

6.1. Programming with an embedded shell language

The combinators are designed based on the typing rules at Figure 1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

```
module Feed (Syntax : Shell.Sym)
{ open Syntax
  let v = observe @@
  { feed |@ Split_by.count(10) |@ nil ;}
}
```

7. Related works

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

Stream programming languages. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Shell languages. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor

[4] a.k.a. character grouping

$$\frac{}{\text{nil } () : \text{Program } T}\ \text{Zero} \qquad \frac{x : \text{Cmd } T \quad x' : \text{Program } T}{x \ ||\% \ x' : \text{Program } T}\ \text{Cons}$$

$$\frac{t : \text{Program } A}{\$\%(t) : \text{Cmd } A}\ \text{Subsection} \qquad \frac{\text{t} : \text{Program A}}{\text{obs}(t) : \text{A}}\ \text{Observe}$$

Figure 1: Typing for the bkhack shell language

incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

8. Further exercises

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

[She] "Shell Command Language." IEEE, The Open Group. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/utilities/V3_chap02.html

[AT&] "AT&T Archives: The UNIX Operating System." [Online]. Available: https://youtu.be/tc4ROCJYbm0?t=296

[Ray03] E. S. Raymond, "Interfaces–Unix Interface Design Patterns," in *The Art of Unix Programming*, 2003, ch. 11, pp. 302–320.

[Pha] T. Phan-Thị, "Phân tích từ vựng," in *Giáo trình Trình biên dịch*, Nhà xuất bản Đại học Quốc gia TP.HCM, ch. 2.6.